Week-7

Aim: Implement data encryption and data decryption.

Description:

In computer networks, data travels across shared physical cables, wireless signals, and intermediate routers. Without protection, an unauthorized third party can intercept packets (eavesdropping/packet sniffing). Cryptography provides **Confidentiality** by ensuring that only the intended receiver can read the transmitted message.

- **Plaintext (P):** The original, readable message generated by the sender (in the code, the plain string).
- **Ciphertext (C):** The scrambled, unreadable message transmitted over the network (in the code, the cipher string). Even if intercepted in transit, it appears as random symbols.
- **Key (K):** A piece of secret information that configures the cryptographic algorithm (in the code, the key length kl).
- **Encryption:** The process of converting Plaintext into Ciphertext before transmission at the Application or Transport layer:

$$C = E_K(P)$$

- **Decryption:** The process performed at the receiver's end to convert Ciphertext back into readable Plaintext:

$$P = D_K(C)$$

Symmetric Key Cryptography

The C program demonstrates **Symmetric Key Cryptography** (Secret-Key Cryptography), where the sender and receiver share the exact same key for both operations.

Plaintext

[Sender] [Receiver]

Plaintext (P) ---> [Encrypt: P ^ K] ---> Ciphertext (C) ---> [Decrypt: C ^ K] ---> Plaintext (P)

▲ ▲

| |

Shared Key (K) Shared Key (K)

At the physical and data-link layers of networks, all data consists of binary bits (0s and 1s). The bitwise XOR operator ($\oplus$) is the fundamental mathematical building block of symmetric encryption because it is **involutory** (self-reversing):

$$(P \oplus K) \oplus K = P$$

Network Protocol Relevance: Modern secure protocols like **TLS/HTTPS**, **SSH**, and **IPsec** use sophisticated symmetric algorithms (such as AES or ChaCha20) for bulk data transmission. These enterprise ciphers expand upon this exact XOR principle by applying multiple rounds of

mathematical mixing and substitution so that unauthorized parties cannot deduce the secret key.

Source Code:

```
#include <stdio.h>
#include <string.h>
int main()
{
   int i, kl;
   char cipher[50], plain[50];
   char key[50];
   printf("Enter plain text: ");
   fgets(plain, sizeof(plain), stdin);
   plain[strcspn(plain, "\n")] = '\0';
   printf("Enter the encryption key: ");
   fgets(key, sizeof(key), stdin);
   key[strcspn(key, "\n")] = '\0';
   kl = strlen(key);
   // Encryption
   for(i = 0; plain[i] != '\0'; i++) {
      cipher[i] = plain[i] ^ kl;
   }
   cipher[i] = '\0';
   printf("Encrypted text is: %s\n", cipher);
   // Decryption
   for(i = 0; cipher[i] != '\0'; i++) {
      plain[i] = cipher[i] ^ kl;
   }
   plain[i] = '\0';
   printf("Decrypted text is: %s\n", plain);
   return 0;
```

}

Output:

1

```
Enter plain text: DATA 2026
Enter the encryption key: lock
Encrypted text is: @EPE$6462
Decrypted text is: DATA 2026
```

2

```
Enter plain text: Attack at dawn!
Enter the encryption key: x
Encrypted text is: @uu`bj!`u!e`vo
Decrypted text is: Attack at dawn!
```